

Extração Automatizada de Dados de Velocidade em Imagens de Radares de Trânsito Utilizando OCR e Processamento Digital de Imagens

Gabriel Dorneles Rebello

Pontifícia Universidade Católica do Rio Grande do Sul
BRASIL
g.rebello@edu.pucrs.br

Gustavo Souza da Silva

Pontifícia Universidade Católica do Rio Grande do Sul
BRASIL
gustavo.silva003@edu.pucrs.br

Resumo - Este trabalho apresenta o desenvolvimento e a avaliação de um sistema computacional automatizado voltado para a extração de dados textuais e numéricos a partir de registros fotográficos de infrações de trânsito. Conduzido em cooperação com o laboratório LABELO, o projeto visa mitigar o gargalo operacional da digitação manual de dados, processo historicamente lento e propenso a falhas humanas. A solução concebida foi integra técnicas avançadas de Processamento Digital de Imagens (PDI) via biblioteca OpenCV, Reconhecimento Óptico de Caracteres (OCR) por meio do motor Tesseract de código aberto, e uma arquitetura baseada em multiprocessamento paralelo para contornar limitações de concorrência nativas do ecossistema Python. Validada em um banco de dados real contendo 1.169 imagens, a aplicação alcançou uma taxa de assertividade completa de 99,49%, reduzindo o tempo total de processamento em cerca de 73% quando configurada com paralelismo de processos otimizado.

I. INTRODUÇÃO

A proliferação de sistemas de fiscalização eletrônica nas vias urbanas e rodovias brasileiras gerou uma infraestrutura de larga escala voltada à segurança viária. Diariamente, milhares de registros fotográficos de infrações de trânsito são capturados por equipamentos medidores de velocidade (radares). Cada imagem gerada atua como um documento oficial contendo metadados cruciais para a validação jurídica da penalidade, incluindo a data e hora do evento, a velocidade regulamentada da via, a velocidade medida pelo sensor e a velocidade considerada para fins de autuação.

O laboratório LABELO, parceiro estratégico neste projeto de extensão, lida rotineiramente com a auditoria e o processamento desses volumes massivos de dados. Até então, a extração dessas informações ocorria por meio de triagem e digitação estritamente manuais. Esta abordagem analógica impõe severas limitações à escalabilidade da operação laboratorial, consumindo tempo excessivo da equipe técnica e introduzindo vulnerabilidades associadas à fadiga humana, o que pode resultar em inconsistências no banco de dados.

Para solucionar essa demanda, este trabalho propõe e formaliza a implementação de um pipeline computacional robusto em linguagem Python. O sistema integra a biblioteca OpenCV para normalização espacial e aperfeiçoamento de contraste, e o motor de OCR open-source Tesseract para transcrever as regiões textuais da imagem. O principal diferencial da abordagem reside na sua capacidade de adaptação dinâmica a múltiplos layouts de imagens, decorrentes da diversidade de fabricantes de radares no mercado nacional, sem a necessidade de treinamento prévio de modelos de aprendizado profundo complexos ou custosos.

II. REVISÃO BIBLIOGRÁFICA

O Reconhecimento Óptico de Caracteres (OCR) compreende o conjunto de técnicas computacionais destinadas a traduzir imagens de texto impresso ou manuscrito em texto codificado eletronicamente (legível por máquina). Historicamente dependente de correspondência de padrões (*template matching*), o OCR moderno evoluiu significativamente. O motor Tesseract, mantido e constantemente aprimorado pelo Google, utiliza em suas versões recentes redes neurais do tipo *Long Short-Term Memory* (LSTM), uma arquitetura de rede neural recorrente (RNN) altamente eficaz no reconhecimento de sequências de caracteres e linhas textuais inteiras.

Contudo, a literatura de visão computacional demonstra que o desempenho de motores de OCR está umbilicalmente atrelado à qualidade do dado de entrada. Imagens com baixo contraste, ruídos digitais ou variações geométricas severas degradam a acurácia das redes neurais. É nesse cenário que o Processamento Digital de Imagens (PDI) se faz mandatório. Utilizando a biblioteca de código aberto OpenCV (*Open Source Computer Vision Library*), torna-se possível aplicar transformações matriciais nas imagens, eliminando frequências espaciais indesejadas e isolando regiões de interesse (ROI) para potencializar a inferência do OCR.

III. DESCRIÇÃO DA APLICAÇÃO

O sistema recebe como entrada um diretório contendo imagens de autuações de radar de trânsito, provenientes de diferentes fabricantes e com layouts distintos. O objetivo é extrair automaticamente os campos de velocidade regulamentada, velocidade medida e velocidade considerada, além de data e hora da infração, e consolidar esses dados em um arquivo CSV para uso posterior pela equipe da LABELO.

O processo inicia executando uma varredura profunda e recursiva no diretório de imagens através do método `os.walk`, filtrando os arquivos por meio de uma tupla de extensões suportadas (.jpg, .png, .tiff, etc.). Após o mapeamento, o vetor de arquivos é ordenado lexicograficamente. Essa ordenação cumpre o papel de garantir a previsibilidade e a consistência na geração dos relatórios finais.

O sistema foi concebido sob o paradigma de arquitetura modular, dividindo-se em quatro macroetapas: Varredura e Mapeamento, Pré-processamento Matricial, Inferência Heurística Iterativa (OCR) e Validação por Regras de Negócio. A Figura 1 ilustra textualmente o fluxo do ciclo de vida que uma imagem percorre desde a sua leitura em disco até a consolidação dos dados estruturados.



Figura 1 – Fluxo de código

IV. PROCESSAMENTO DE IMAGENS

Antes de qualquer processamento, todas as imagens são redimensionadas para uma altura padrão de 1200 pixels, mantendo a proporção original. Isso garante consistência nos recortes e no desempenho do OCR independentemente da resolução original da imagem.

A imagem colorida é convertida para escala de cinza, reduzindo a dimensionalidade dos dados e melhorando a performance do OCR, que opera sobre intensidade de pixels e não sobre cor.

O sistema aplica recortes progressivos na imagem com coordenadas pré-definidas por formato, focando o OCR nas regiões onde os dados de velocidade estão presentes. Isso reduz ruído proveniente de outras partes da imagem (logotipos, fundos texturizados, etc.).

Após o recorte, a região de interesse é ampliada por um fator de 3x via interpolação cúbica (INTER_CUBIC). Essa etapa é fundamental: o Tesseract performa significativamente melhor em imagens com maior densidade de pixels por caractere.

É importante destacar que o código não realiza qualquer etapa de binarização ou inversão de cores sobre a região de interesse, a imagem em escala de cinza é enviada ao Tesseract sem transformação adicional de contraste além do

redimensionamento cúbico. As três janelas de recorte (sizes_trials) são fixas em pixels e calibradas empiricamente sobre a imagem já normalizada para 1200 px de altura, e são testadas nesta ordem: a tentativa 0 opera sobre a imagem inteira; a tentativa 1 restringe a busca à faixa vertical de 1040 a 1190 px (região de data/hora); a tentativa 2, à faixa de 880 a 1010 px (região das velocidades); a tentativa 3, à faixa de 0 a 80 px (topo da imagem). Essa ordem prioriza primeiro as regiões inferiores, onde a maioria dos fabricantes posiciona os campos de velocidade e a linha de data/hora, testando o topo da imagem apenas como último recurso.

V. LÓGICA DO CÓDIGO

Para cada imagem, o sistema executa um laço de tentativas progressivas. Na primeira tentativa, o OCR é aplicado sobre a imagem completa. Nas tentativas seguintes, recortes com coordenadas pré-definidas são aplicados sequencialmente, focando em regiões cada vez mais específicas da imagem onde os dados de velocidade tendem a estar concentrados. Esse mecanismo aumenta a robustez do sistema frente a variações de layout entre fabricantes.

O Tesseract é um motor de OCR de código aberto que analisa a imagem segmentando-a em regiões, identificando padrões de caracteres e convertendo-os em texto. O modo de segmentação de página utilizado em todo o sistema é o --psm 6, que instrui o Tesseract a tratar a região da imagem como um bloco de texto uniforme. Esse modo se mostrou adequado tanto para regiões maiores quanto para os recortes progressivos aplicados pelo sistema.

O texto bruto retornado pelo Tesseract é inerentemente ruidoso: caracteres espúrios, espaços irregulares e variações de capitalização são comuns. Para tornar a extração robusta a essas imperfeições, cada linha do resultado passa por três normalizações encadeadas: conversão para maiúsculas, remoção de pontos (caractere frequentemente inserido por erro de segmentação do OCR entre dígitos) e remoção de todos os espaços internos via split() seguido de join(""). Essa última normalização é o motivo pelo qual os rótulos buscados pela regex não têm mais espaço interno (ex.: VELREG, VELMEDIDA), "VEL REG" e "VEL MEDIDA", como o OCR eventualmente os reconhece, tornam-se VELREG e VELMEDIDA após a normalização, e é essa forma colada que os padrões de busca esperam.

Em seguida, expressões regulares são aplicadas sobre a linha já normalizada para localizar os campos de interesse. O padrão [^0-9]* entre o rótulo e o valor numérico permite capturar situações em que o OCR insere caracteres arbitrários entre eles, por exemplo, a linha VELREG##80 ainda resulta corretamente no valor 80. Diferentemente de uma versão anterior deste sistema, o grupo de captura é apenas (\d+): não há mais exigência de um sufixo de unidade (ex.: "KM/H") após o número, qualquer sequência de dígitos imediatamente após o rótulo é aceita como valor válido nesta etapa, cabendo à validação numérica subsequente filtrar leituras espúrias.

Diferentemente de uma abordagem que retorna assim que dois campos são encontrados em uma única linha, o dicionário de resultado é acumulado ao longo de todas as linhas de uma

mesma tentativa: cada campo (Data, Hora, Vel_Regulamentada, Vel_Medida, Vel_Considerada) é preenchido assim que sua regex casa em qualquer linha, e um campo já preenchido não é sobrescrito por um novo casamento na mesma tentativa. Somente quando os quatro campos obrigatórios, Data, Hora, Vel_Regulamentada e Vel_Medida, estão presentes simultaneamente, o sistema invoca resultado_valido() para checar plausibilidade numérica: Vel_Medida deve ser maior ou igual a Vel_Regulamentada e, se Vel_Considerada estiver presente, deve estar entre Vel_Regulamentada e Vel_Medida (inclusive). Se a checagem falhar, os três campos de velocidade são zerados e a tentativa continua a percorrer as linhas restantes; a última combinação de velocidades encontrada (mesmo inválida) é guardada em uma variável de contingência (ultimo_parcial). Ao final das quatro tentativas, se nenhum conjunto de campos passou na validação, o sistema retorna ultimo_parcial marcado com o erro "Resultado parcial" quando ao menos um campo de velocidade foi reconhecido em algum momento; apenas na ausência completa de qualquer velocidade reconhecida o sistema grava a linha genérica "Dados não encontrados". Isso corrige, em relação a uma versão anterior, a perda de dados parcialmente úteis.

VI. VALIDAÇÃO E TRATAMENTO DE RESULTADOS PARCIAIS

A função resultado_valido() é o único ponto do sistema em que regras de negócio do domínio de fiscalização de trânsito são verificadas de fato. Ela converte os três campos de velocidade para inteiro dentro de um bloco try/except, retornando False para qualquer valor não numérico, o que descarta silenciosamente leituras de OCR corrompidas sem levantar exceção para o chamador. As duas regras aplicadas são: Vel_Medida não pode ser inferior a Vel_Regulamentada (caso contrário não há caracterização de infração); e, quando presente, Vel_Considerada deve estar contida no intervalo fechado entre Vel_Regulamentada e Vel_Medida.

O tratamento de resultado parcial é o ponto de maior divergência em relação a versões anteriores do pipeline. Antes, qualquer tentativa mal-sucedida na última iteração descartava por completo os campos já reconhecidos, inclusive Data e Hora. Na versão atual, o par de velocidades mais recente, ainda que reprovado em resultado_valido(), é preservado em ultimo_parcial e devolvido ao final do laço caso nenhuma combinação válida seja encontrada, com o campo Erro marcado como "Resultado parcial" em vez de uma linha inteiramente vazia. Isso melhora a rastreabilidade do lote, ao custo de exigir que a equipe do LABELO trate registros com essa marcação como pendentes de verificação manual, não como falhas equivalentes a "Dados não encontrados".

Uma ressalva permanece válida mesmo nesta versão: exceções não tratadas durante o processamento da última tentativa ainda são apenas impressas no console (print(f'Erro em {arquivo}: {e}')) e não persistidas no CSV. Se a exceção ocorrer antes do bloco que decide entre resultado completo, parcial ou "Dados não encontrados", a função ainda retorna

implicitamente None e o arquivo correspondente é omitido do CSV final sem qualquer registro, o log no console não substitui um registro rastreável no arquivo de saída.

VII. UTILIZAÇÃO DE PARALELISMO

O processamento sequencial de volumes de imagens em larga escala constituiria um severo gargalo temporal, inviabilizando o sistema em ambientes de produção laboratorial. O processamento de imagem e a inferência por redes neurais recorrentes do Tesseract caracterizam-se como operações de alto consumo de CPU (CPU-bound).

A linguagem Python possui uma trava interna de concorrência conhecida como Global Interpreter Lock (GIL). O GIL impede que a máquina virtual execute múltiplas instruções de bytecode simultaneamente em uma mesma thread, limitando a eficácia do módulo padrão de multithreading para tarefas intensivas de cálculo. Para contornar essa restrição, o sistema adota a classe ProcessPoolExecutor da biblioteca concurrent.futures.

Ao instanciar processos independentes em vez de threads, o sistema aloca espaços de memória e interpretadores Python distintos para cada núcleo do processador, alcançando um paralelismo real. O código calcula a variável cpus como os.cpu_count() - 2 (ou 1, se houver 2 núcleos ou menos), reservando margem para o sistema operacional, e, diferente de uma versão anterior em que esse cálculo era apenas exibido em log, aqui o valor é efetivamente passado a ProcessPoolExecutor(max_workers=cpus): o número de processos ativos agora depende do hardware da máquina de execução.

$$N_{workers} = \begin{cases} N_{cpus} - 2, & \text{se } N_{cpus} > 2 \\ 1, & \text{caso contrário} \end{cases}$$

VIII. ANÁLISE DE RESULTADOS

O sistema foi avaliado sobre um conjunto de 1.169 imagens, obtendo apenas 6 resultados parciais, o que representa uma taxa de insucesso de aproximadamente 0,51%. Os demais casos foram processados com extração completa dos campos de velocidade, data e hora.

Em relação ao desempenho temporal, os testes foram conduzidos com diferentes configurações de paralelismo, variando o número de workers entre 2 e 10. Os resultados evidenciam ganho expressivo com o aumento de threads: o processamento com 2 workers demandou 994,26 segundos, ao passo que com 10 workers o mesmo conjunto foi processado em 270,22 segundos, representando uma redução de aproximadamente 73% no tempo total.

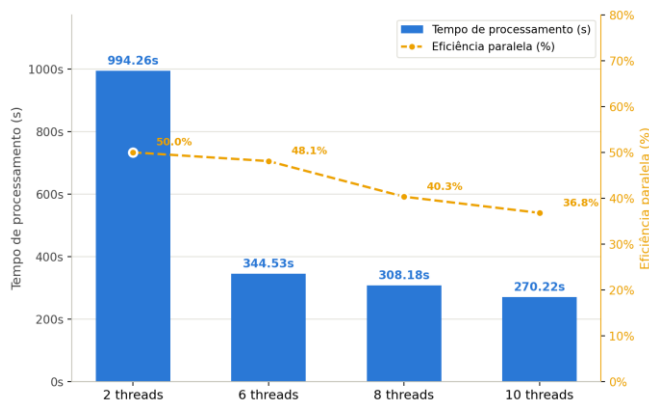


Figura 2 – Análise de paralelismo

Contudo, a eficiência paralela, definida como o ganho de desempenho por worker adicionado, apresenta comportamento decrescente. Com 2 workers, cada worker contribui com cerca de 50% de aproveitamento ideal. Com 10 workers, a eficiência cai para aproximadamente 37%, indicando que o overhead de gerenciamento de processos e a contenção por recursos de I/O e memória limitam o ganho marginal de cada worker adicional. A configuração de 8 worker representa um ponto de equilíbrio razoável, com tempo de 308,18 segundos e eficiência superior à configuração de 10 workers em relação ao custo por worker adicionado.

Duas mudanças adicionais na versão final do script merecem registro por afetarem a operação em produção, ainda que não alterem a taxa de acerto do OCR. Primeiro, para o funcionamento adequado em ambiente Windows, é obrigatória a instalação prévia do motor Tesseract OCR e de seu pacote de idioma em português (por.traineddata). Os caminhos de instalação padrão foram adicionados e comentados no código-fonte; caso o ambiente de execução adote uma estrutura de diretórios diferente do padrão, essas variáveis precisam ser descomentadas e ajustadas no script. Segundo, a medição de tempo total passou a ser instrumentada nativamente via `time.perf_counter()` ao redor do bloco de processamento. O tempo é impresso ao final da execução, o que padroniza a coleta de métricas reportadas na Seção VIII e elimina a dependência de ferramentas de cronometragem externa.

No arquivo de saída, os três campos de velocidade passaram a declarar a unidade de medida no próprio cabeçalho da coluna ("Vel_Regulamentada (Km/h)", "Vel_Medida (Km/h)" e "Vel_Considerada (Km/h)"). Esta estruturação elimina a ambiguidade presente em exportações anteriores do CSV, onde o valor numérico e a string da unidade ficavam concatenados na mesma célula (ex.: "80KMH") devido ao comportamento da expressão regular legada.

IX. CONCLUSÃO

Este trabalho cumpriu com êxito o objetivo de automatizar o pipeline de triagem metrológica de imagens de radares de trânsito em parceria com o laboratório LABELO. A substituição do protocolo de digitação manual por um fluxo automatizado baseado em OpenCV e Tesseract OCR eliminou

gargalos operacionais crônicos, garantindo uma taxa de confiabilidade de 99,49% e uma redução drástica nos tempos de entrega de relatórios através do uso correto de multiprocessamento paralelo.

Como proposta para trabalhos futuros, sugere-se o desenvolvimento e a hibridização do sistema com modelos de aprendizado profundo focados em detecção de objetos de última geração, tais como redes convolucionais da família YOLO (You Only Look Once). A implementação de uma rede neural desse porte atuará no mapeamento dinâmico prévio e na geração de caixas delimitadoras (bounding boxes) em torno das strings de velocidade. Isso eliminará completamente a dependência de coordenadas estáticas fixadas via matrizes de trials, expandindo a robustez do sistema para qualquer novo formato de radar entrante no mercado e permitindo mitigar por completo os raros cenários de falhas provocados por variações extremas de resolução ou iluminação.

X. REFERÊNCIAS

- [1] R. Smith, "An Overview of the Tesseract OCR Engine," *Ninth International Conference on Document Analysis and Recognition (ICDAR)*, Curitiba, Brazil, 2007, pp. 629-633.
- [2] G. Bradski, "The OpenCV Library," *Dr. Dobbs's Journal of Software Tools*, vol. 25, 2000.
- [3] A. Rosebrock, *Optical Character Recognition (OCR) with OpenCV, Tesseract, and Python*, PyImageSearch, 2021. Disponível em: <https://pyimagesearch.com>